

Experiment 11: Heap Sort

Aim

To implement the **Heap Sort** algorithm using Python to sort a list of elements in ascending order.

Algorithm

Step 1: Start.

Step 2: Read the number of elements n.

Step 3: Read the array elements.

Step 4: Build a **Max Heap** from the array.

Step 5: Swap the first element (largest) with the last element.

Step 6: Reduce the heap size by one and heapify the root.

Step 7: Repeat Steps 5 and 6 until the array is sorted.

Step 8: Display the sorted array.

Step 9: Stop.

Source Code (Python)

```
main.py
1 # Heap Sort
2
3 def heapify(arr, n, i):
4     largest = i
5     left = 2 * i + 1
6     right = 2 * i + 2
7
8     if left < n and arr[left] > arr[largest]:
9         largest = left
10
11     if right < n and arr[right] > arr[largest]:
12         largest = right
13
14     if largest != i:
15         arr[i], arr[largest] = arr[largest], arr[i]
16         heapify(arr, n, largest)
17
18 def heap_sort(arr):
19     n = len(arr)
20
21     # Build Max Heap
22     for i in range(n // 2 - 1, -1, -1):
23         heapify(arr, n, i)
24
25     # Extract elements one by one
26     for i in range(n - 1, 0, -1):
27         arr[0], arr[i] = arr[i], arr[0]
28         heapify(arr, i, 0)
29
30 n = int(input("Enter number of elements: "))
31
32 arr = []
33
34 print("Enter the elements:")
35 for i in range(n):
36     arr.append(int(input()))
37
38 heap_sort(arr)
39
40 print("Sorted array:", arr)
```

Output

```
Output
Enter number of elements: 6
Enter the elements:
40
10
30
50
20
60
Sorted array: [10, 20, 30, 40, 50, 60]

=== Code Execution Successful ===
```

Time Complexity

- Best Case: $O(n \log n)$
- Average Case: $O(n \log n)$
- Worst Case: $O(n \log n)$

Space Complexity

- $O(1)$ (excluding recursion stack)

Result

The **Heap Sort** algorithm was implemented successfully in Python. The program builds a **Max Heap**, repeatedly extracts the largest element, and sorts the array in ascending order. The output was obtained correctly.